Language over demo breadcrumbs for a frozen LeWorldModel on Push-T

Alexander Bashirov Didar Seyidov Faith Ogunwoye
Skoltech

19 October 2026

Abstract

The public Push-T checkpoint of LeWorldModel stays frozen. Its planner reaches a goal image that is about 25 demonstration steps ahead, and it fails when the goal is a long live task or an in-place turn. We build a graph on expert demonstrations and hand the planner a nearby frame from that graph, then replan. On held-out dataset goals the breadcrumb planner reaches 34/50 at offset 50 and 29/50 at offset 100, against 24/50 and 10/50 for the same planner aimed at the final frame. A small language model maps a sentence onto one of seven commands, and the graph supplies the goal. On 111 held-out sentences, gold commands succeed on 57 and parsed commands on 51. A network that predicts the next latent from the sentence, with no graph at test time, succeeds on 18 of those 111.

1 Introduction

Push-T asks a pusher to move a T-shaped block onto a target. LeWorldModel (LeWM) encodes a camera frame as a 192-dimensional latent and predicts how that latent changes under an action [Maes et al., 2026]. The checkpoint used here, `quentinll/lewm-pusht`, was not trained with language. The course question is whether words can name a goal that this frozen planner can already follow.

A short goal from the demonstration set is followable. A goal taken from a random reset is not, under the horizon and budget that were tested. An in-place rotation is not either, even when the goal image is a real render of that rotation. The method in this note does not invent a goal image from text. It looks through recorded expert demonstrations, finds a route of real states, and asks the existing planner to chase the next state on that route.

2 Earlier measurements

The encoder stays frozen. A ridge map from the latent to $(\cos\theta, \sin\theta)$, with episodes held out, has circular mean absolute error 12.08 degrees on scripted 180° frames and 14.58 degrees on frames within 30 degrees of angle 0. Those two slices are reported in `reports/lewm_language_port_probe.tex`. A constant latent displacement per command string has residual ratio 0.991 against the root-mean-square of the observed change, so a lookup table of text vectors does not explain the latent step.

Planning uses cross-entropy method search: 300 samples, 30 iterations, 30 elites, horizon 5, and an action block of 5. Actions are z-scored with the Push-T training column, then inverse-transformed and clipped to $[-1, 1]$ before the environment step. With that transform, image-goal planning on dataset rows 25 steps apart succeeds on 45 of 50 episodes at a budget of 50 environment steps. An earlier planner that multiplied actions by 5, without those statistics, succeeded on 0 of 50 and is not a baseline.

The same planner does not spin the block in place. A cost written on the angle probe reaches a +90° turn on fraction 0.10 of 20 held-out starts and a 180° turn on fraction 0.00. Giving the planner the encoding of a rendered in-place target does not raise that rate: reach is 0.00 for +90° at budget 50, and 0.00 for +180° at budgets 50 and 100. A live goal from a random reset, chased with the official action transform, succeeds on 0 of 20 episodes at budget 50. Three rendered stages at +30°, +60°, and +90° pass the in-place gate on 1 of 20 starts. These numbers are the ones in `reports/lewm_mmagai_status.tex`, `reports/lewm_oracle_rotate_cem.tex`, and `research/LEDGER.md`. They were not recomputed for this note.

3 Method

The planner is reliable about 25 steps out and weak farther away. Caselli et al. [2026] report that future frames from the demonstration the start came from work as subgoals, at 73.3 percent at offset 50 under their protocol. A new situation has no demonstration of its own. The graph used here is built on other demonstrations.

The breadcrumb script reuses the official planner. One node is stored every 5 demonstration steps, on the train split only (90 percent of episodes, split seed 0). Forward edges follow a demonstration. Jump edges link nodes from different demonstrations closer than 15 in a state distance: agent position and block position in pixels, and angle as 60 pixels per radian. Retrieval uses the environment state, not the latent.

Each hop finds the node near the current state with the cheapest route to the goal, walks about 25 demonstration steps along that route, renders that state, and runs one plan of 25 environment steps toward its encoding. The

Table 1: Breadcrumbs against the flat planner. Budget 300 except the setup row, which is the official protocol at budget 50.

Task	n	Flat	Crumbs
Official image goal, offset 25, budget 50	50	47	—
Dataset goal, offset 50	50	24	34
Dataset goal, offset 100	50	10	29
Live random goal	20	0	3
Word templates	64	18	34

graph is read again from the new state. When the route reaches the goal, the planner chases the goal frame. The flat baseline is the same loop with no graph, at the same budget. Scored runs on the lab RTX 5090 use a budget of 300 environment steps. The setup check, official image-goal CEM at budget 50, scored 47/50. The earlier 4090 run of that same protocol scored 45/50.

Words pick a region rather than a frame. Seven templates are scored: put the T on the target, turn it 90 degrees clockwise, turn it 90 degrees counterclockwise, and push it left, right, up, or down by about 80 pixels. Screen y points down, so "up" is $-y$. A positive block angle turns the T clockwise on screen, so clockwise is $\theta_0 + 90°$. Success uses the environment tolerances: 20 pixels and 20 degrees. Turns allow 40 pixels of drift. A push whose target leaves the box $[80, 432]$ is dropped before the run.

A sentence is mapped to one of those commands by Qwen2.5-1.5B-Instruct, run locally and greedily, with a keyword baseline built only from the seven templates. Both the gold command and the parsed command are scored on the gold region. A second head, `scripts/latent_subgoal.py`, takes the start latent, the current latent, and a frozen Qwen embedding and predicts the next latent. At test time that head replaces the graph and the parser. The 120 held-out sentences were written and committed before either language evaluation ran.

4 Results

Table 1 is the breadcrumb comparison. At offset 50, crumbs reach 34/50 and flat reaches 24/50. At offset 100, crumbs reach 29/50 and flat reaches 10/50. The dry graph finds a route for 50/50 of the offset-50 tasks, with median 0.5 jumps. On live resets the dry graph finds a route for 13/20. Crumbs then reach 3/20 and flat reaches 0/20. Median block error on that live set falls from 162 pixels to 58 pixels.

Word templates, on live resets with seeds 0–9, give 64 command and start pairs after 6 pushes are dropped. The dry graph has a route for 56/64. Crumbs succeed on 34/64 and flat on 18/64. By command, crumbs score

Table 2: Held-out sentences. Nine pushes are excluded because the gold destination leaves the arena, so execution is scored on 111 sentences. Parsing is scored on all 120.

Measurement	Count	Rate
Qwen parse correct	101/120	0.842
Keyword parse correct	31/120	0.258
Gold command, crumbs	57/111	0.514
Parsed command, crumbs	51/111	0.459
Subgoal head	18/111	0.162
Subgoal head, shuffled text	10/111	0.090

target 6/10, clockwise 5/10, counterclockwise 3/10, left 4/8, right 4/8, up 7/10, and down 5/8. Turns alone are 8/20 for crumbs and 4/20 for flat. Under the stricter in-place gate, drift below 15 pixels, crumbs score 0/20 and flat scores 1/20. The turns that succeed move the block by 15–40 pixels. The demonstrations do contain in-place turns: 8,040 of 427,396 nodes, in 1,691 of 16,817 episodes, under the rule that the block moves less than 15 pixels while turning more than 90 degrees within 50 demonstration steps.

Table 2 is the language evaluation. The parser is correct on 101 of 120 sentences. By style the counts are plain 30/32, casual 25/29, indirect 18/24, and odd 28/35. By action they are target 25/30, turn 35/45, and push 41/45. The keyword baseline is correct on 31 of 120. No gold label was marked invalid, and no label was disputed.

End to end, gold commands succeed on 57 of 111 eligible sentences and parsed commands on 51 of 111. The gap is 5.4 percentage points. Ninety parsed commands match the gold command and reuse that run. Paired outcomes are 8 gold-only successes and 2 parsed-only successes. By action, gold is target 23/30, turn 18/45, and push 16/36. Parsed is target 20/30, turn 17/45, and push 14/36. Nine pushes are excluded because the gold destination falls outside the arena.

The subgoal head has validation mean squared error 122.9 against 184.7 for copying the current latent, a ratio of 0.67. Shuffled text on that validation set has error 158.5. On the 111 sentences the head succeeds on 18, against 51 for the parser plus the graph and 57 for gold commands plus the graph. With each sentence given another row's text, the head succeeds on 10. The gain is concentrated on the target command: 11/30 with the sentence, 2/30 with shuffled text. Turns are 3/45 in both conditions. Pushes are 4/36 with the sentence and 5/36 with shuffled text. Train error is 39 and validation error is 123, so the head overfits.

5 Discussion

Breadcrumbs help when the goal is farther than one planning hop and a route exists in other demonstrations. The gain grows from offset 50 to offset 100. On a random live goal the graph often has no nearby route, and the scored gain is 3 episodes. Word templates sit between those two regimes: the graph finds a route for most pairs, and crumbs beat flat by 16 episodes on 64 pairs.

Parsing is not the main loss. The parser misses 19 of 120 sentences, and the end-to-end gap between gold and parsed commands is 6 episodes on 111. Gold commands themselves succeed on 57 of 111. The remaining miss is execution. Turns that the template scorer accepts are not in-place turns. The in-place gate stays at 0/20 for crumbs, which matches the earlier oracle result that a rendered in-place image is not followed.

Predicting the next latent from the sentence, without the graph, is weaker than retrieving a demonstration frame. The head beats a copy of the current latent on the validation set, and the target command is sensitive to the words. Turns and pushes, which depend on the start pose, are not. The parser plus the graph remains about three times as successful on the same 111 sentences.

The published hierarchical numbers are a different protocol. Caselli et al. [2026] report flat LeWM at 52.7 percent at offset 50 after a hyperparameter sweep, and oracle subgoals from the same demonstration at 73.3 percent. The local flat run at offset 50 is 24/50, and the local crumbs run is 34/50. Those pairs should be read inside one protocol. The 4090 image-goal rate of 45/50 and the 5090 setup rate of 47/50 are the local check that the short-horizon planner still matches.

6 Conclusion

A frozen LeWM planner follows a nearby demonstration frame and does not follow a long or in-place goal under the tested budget. Routes through other demonstrations raise success from 24/50 to 34/50 at offset 50 and from 10/50 to 29/50 at offset 100. Sentences can name those routes: a 1.5B parser is correct on 101/120 held-out lines, and parsed commands lose 6 successes against gold commands on 111 eligible tasks. A latent subgoal head without the graph reaches 18/111. The encoder was not finetuned.

References

Niccolò Caselli, Francesco Massafra, Samuele Punzo, Salvatore Lo Sardo, Ippokratis Pantelidis, and Sathya Kamesh Bhethanabhotla. Mind the gap: Promises and pitfalls of hierarchical planning in LeWorldModel. *arXiv preprint arXiv:2607.12547*, 2026. URL https://arxiv.org/abs/2607.12547.

Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. URL https://arxiv.org/abs/2603.19312.